

2-Mark Questions & Answers — Data Structures Module 1

Topic 1: Concept of ADT

Q1. Define Abstract Data Type (ADT).

An **Abstract Data Type (ADT)** is a mathematical model for data types where a data type is defined by its behavior (semantics) from the point of view of a user of the data, specifically in terms of possible values, possible operations on data of this type, and the behavior of these operations. It specifies **what** operations are to be performed, but not **how** these operations will be implemented.

Q2. What is the difference between data type and data structure?

- **Data Type:** A classification that specifies which type of value a variable can hold (e.g., `int`, `float`, `char`). It is built into the language.
 - **Data Structure:** A specialized format for organizing, processing, retrieving, and storing data (e.g., Arrays, Linked Lists, Trees) along with algorithms to manipulate it efficiently.
-

Q3. List any two examples of ADTs.

1. **Stack ADT:** Operates on LIFO (Last In First Out) principle with operations `push()`, `pop()`, `peek()`.
 2. **Queue ADT:** Operates on FIFO (First In First Out) principle with operations `enqueue()`, `dequeue()`.
-

Topic 2: Types of Data Structures

Q4. What is a linear data structure? Give two examples.

A **Linear Data Structure** is a data structure in which elements are arranged sequentially or linearly, where each element is connected to its previous and next adjacent elements.

- **Examples:** Arrays, Linked Lists, Stacks, Queues.

Q5. What is a non-linear data structure? Give two examples.

A **Non-Linear Data Structure** is a data structure in which data elements are not arranged sequentially or linearly. Elements are connected in a hierarchical or network relationship.

- **Examples:** Trees, Graphs.

Q6. Differentiate between static and dynamic data structures (any two points).

Feature	Static Data Structure	Dynamic Data Structure
Memory Allocation	Allocated at compile time.	Allocated at runtime dynamically.
Size Flexibility	Fixed size, cannot change during execution (e.g., Array).	Flexible size, grows or shrinks as needed (e.g., Linked List).

Topic 3: Operations on Data Structures

Q7. List any four basic operations performed on data structures.

1. **Traversing:** Accessing each record exactly once.
2. **Searching:** Finding the location of a target element.
3. **Insertion:** Adding a new element to the structure.
4. **Deletion:** Removing an existing element from the structure.

Q8. Define "Traversal" operation on a data structure.

Traversal is the process of visiting (accessing and processing) every element in a data structure exactly once in a systematic order to perform an operation like printing or updating.

Q9. What is meant by "searching" in data structures?

Searching is the process of finding the location or index of a specific target element within a data structure. Common searching techniques include **Linear Search** ($O(n)$) and **Binary Search** ($O(\log n)$).

Topic 4: Arrays & Multi-Dimensional Arrays

Q10. Define an array. What is its base address?

- **Array:** A linear collection of homogeneous (same data type) data elements stored in contiguous memory locations.
- **Base Address:** The physical memory address of the very first element (index 0) of the array, denoted as **BA** or **Loc(A[0])**.

Q11. Give the formula for calculating the address of an element in a 1-D array.

The address of element **A[i]** in a 1-D array is calculated as:

Equation:

$$\text{Address of } A[i] = B + w \times (i - LB)$$

Where:

- **B** = Base Address
- **w** = Size of each element in bytes
- **i** = Index of the desired element
- **LB** = Lower Bound (usually 0)

Q12. What is a 2-D array? Give its declaration syntax in C.

A **2-D Array** (Two-Dimensional Array) is an array of arrays organized in a matrix form consisting of rows and columns.

- **Syntax in C:** `data_type array_name[row_size][column_size];`
 - **Example:** `int matrix[3][4];` (declares a 3×4 integer matrix).
-

Topic 5: String Manipulation

Q13. Define a string. How is it represented in memory?

A **String** is a sequence of characters terminated by a special null character `'\0'`.

- **Memory Representation:** Stored as a contiguous 1-D array of characters where the last byte holds the ASCII value `0` (`'\0'`).
 - **Example:** `"DATA"` is stored as `['D', 'A', 'T', 'A', '\0']`.
-

Q14. List any four string handling functions in C.

1. `strlen(s)` — Returns the length of string `s`.
 2. `strcpy(dest, src)` — Copies string `src` to `dest`.
 3. `strcat(dest, src)` — Appends string `src` to `dest`.
 4. `strcmp(s1, s2)` — Compares strings `s1` and `s2` lexicographically.
-

Q15. What is the difference between `strcpy()` and `strcat()` ?

- `strcpy(dest, src)` overwrites the destination array `dest` with the contents of `src`.
 - `strcat(dest, src)` preserves the existing content of `dest` and appends `src` to the end of `dest`.
-

Topic 6: Self-Referential Structures with Pointers

Q16. Define a self-referential structure. Give its syntax.

A **Self-Referential Structure** is a structure containing one or more structure members that are pointers pointing to a structure of the same type.

```
struct Node {
    int data;
    struct Node *next; // Self-referential pointer
};
```

Q17. What is the use of a NULL pointer in a self-referential structure?

A **NULL pointer** (`NULL` or `0`) in a self-referential structure is used to indicate the end of a dynamic sequence (such as the end node of a linked list or an unlinked child in a binary tree node), preventing invalid memory dereferencing.

Q18. Give an example declaration of a node in a linked list.

```
struct Node {  
    int data;          // Data part  
    struct Node *next; // Address part pointing to next node  
};
```